RL Autonomous Racing Agent

Weekly Progress Report — Week 5

Reward Engineering, Physics-Based Speed Control, and Full Training Run

1. Objective

Last week's model (PPO + Stable-Baselines3 on Gymnasium CarRacing-v3) trained successfully in terms of raw reward, but the resulting driving behavior was poor: the agent cornered aggressively and accelerated excessively, with no sense of momentum or when to slow down. This week's goal was to diagnose why the reward signal wasn't discouraging that behavior, redesign the reward function around real driving physics, and retrain a model that drives smoothly and confidently.

2. Reward Function Iteration

The core insight driving this week's work: PPO optimizes exactly what the reward function measures. If the reward doesn't distinguish between 'fast on a straight' and 'fast into a corner,' the agent has no reason to brake. Getting a model that actually corners well required rebuilding the reward function in stages, testing each one before moving to the next.

2.1 Baseline: Steering Jerk Penalty

The first version penalized large frame-to-frame changes in the steering action, and separately penalized throttle input while steering was sharp. This helped marginally but didn't address the underlying issue: the agent still had no concept of speed relative to the upcoming turn, so it could floor the throttle right up to the corner and only then react.

2.2 Speed-Aware Penalty

The next version multiplied the penalty by current speed whenever the steering angle was sharp, on the reasoning that fast + sharp-turn should cost more than slow + sharp-turn. This was a step forward but still reactive — it only evaluated the current instant, not whether the car had enough time/distance to actually slow down before the corner arrived.

2.3 Curvature-Aware Physics Model

The reward function was redesigned around an actual physics relationship used in real racing lines: the maximum safe speed for a corner of radius r is approximately $v_{max} = \sqrt{\mu \cdot g \cdot r}$, where μ is an effective friction coefficient and g is gravity. This required:

- Locating the car's position on the track's internal centerline data each step
- Estimating the curvature of the upcoming track segment using a lookahead window and the Menger curvature formula (three-point curvature from cross product / side lengths)
- Converting curvature into a target safe speed via the formula above, and penalizing the car for exceeding it — this made the signal anticipatory rather than reactive

This surfaced a real bug: early tests produced episode rewards in the range of −10,000 to −78,000, far outside CarRacing-v3's normal −100 to +900 range. The squared overspeed penalty was compounding unchecked due to a unit mismatch between Box2D's internal physics scale and the real-world friction/gravity assumption in the formula. Fixed by capping the per-step penalty and tuning the friction coefficient empirically rather than assuming a literal physical value.

2.4 Mechanical Constraints (Guaranteed, Not Learned)

Two fixes were added that don't rely on the policy learning anything — they constrain the action directly, so the effect is immediate rather than requiring training time:

- Steering rate limiting: capped how much the steering angle can change in a single step, physically preventing violent wheel-snaps regardless of what the policy outputs.
- Wrong-direction detection: compared the car's velocity vector against the track's local tangent direction; if consistently opposed for more than a set number of steps, the episode terminates early instead of wasting rollout time driving backward.

2.5 Braking: From Hard Cutoff to Graduated Control

The first working version forced full throttle-cut and hard braking the instant the car exceeded the curve's safe speed. This worked but felt unnatural — braking was abrupt, sometimes triggered too early (lookahead distance was too long), and occasionally over-slowed corners that didn't need it. The final version replaced the binary cutoff with a proportional braking model derived from the standard braking-distance equation:

$$v^2 = v_0^2 - 2 \cdot a \cdot d$$

Given current speed, target safe speed, and distance remaining to the corner, this computes the exact deceleration required to reach the corner at a safe speed, then expresses it as a fraction of the car's maximum braking capability. Throttle is scaled down proportionally instead of cut to zero, and brake is blended with (not simply replaced by) the policy's own output. This produced visibly gradual, natural-feeling deceleration into corners rather than an on/off cutoff.

Tuning this stage involved iterating on three interacting parameters — friction coefficient (how conservative the safe-speed estimate is), braking power (how strong the forced braking is), and lookahead distance (how early the car reacts) — adjusting one at a time and re-checking behavior via short recorded rollouts before moving on.

3. Infrastructure Debugging

Several environment/framework issues surfaced during setup and had to be resolved before training could run reliably:

- VecNormalize synchronization: Stable-Baselines3's EvalCallback requires the evaluation environment to be wrapped in VecNormalize identically to the training environment, or it fails when trying to sync running statistics between the two.
- Entropy coefficient scheduling: PPO's learning_rate supports a schedule function, but ent_coef does not — passing a schedule function there causes a type error during the loss calculation. Resolved by using a fixed entropy coefficient instead.
- Missing dependencies: stable-baselines3, moviepy (required by Gymnasium's RecordVideo wrapper for video encoding), and box2d support all had to be installed before the pipeline would run end-to-end.

4. Validation Methodology

Rather than committing to a full 2,000,000-timestep run after every reward function change, each iteration was first validated with a short 10,000-timestep smoke test. This confirmed the full pipeline — training step, reward computation, normalization sync, checkpointing, evaluation, and video recording — ran cleanly end-to-end in about a minute, and let actual driving behavior be checked via recorded demo clips before committing hours of compute to a change that might need further tuning. This approach caught several issues (the reward blow-up, the eval-env wrapper mismatch, over-aggressive braking) cheaply, before they could waste a full training run.

5. Final Training Run

With the physics-based reward function, mechanical steering/wrong-way constraints, and graduated braking model in place, a full training run of 2,000,000 timesteps was completed using PPO with a CNN policy, 4-stack frame stacking, 4 parallel environments, and a linearly decaying learning rate.

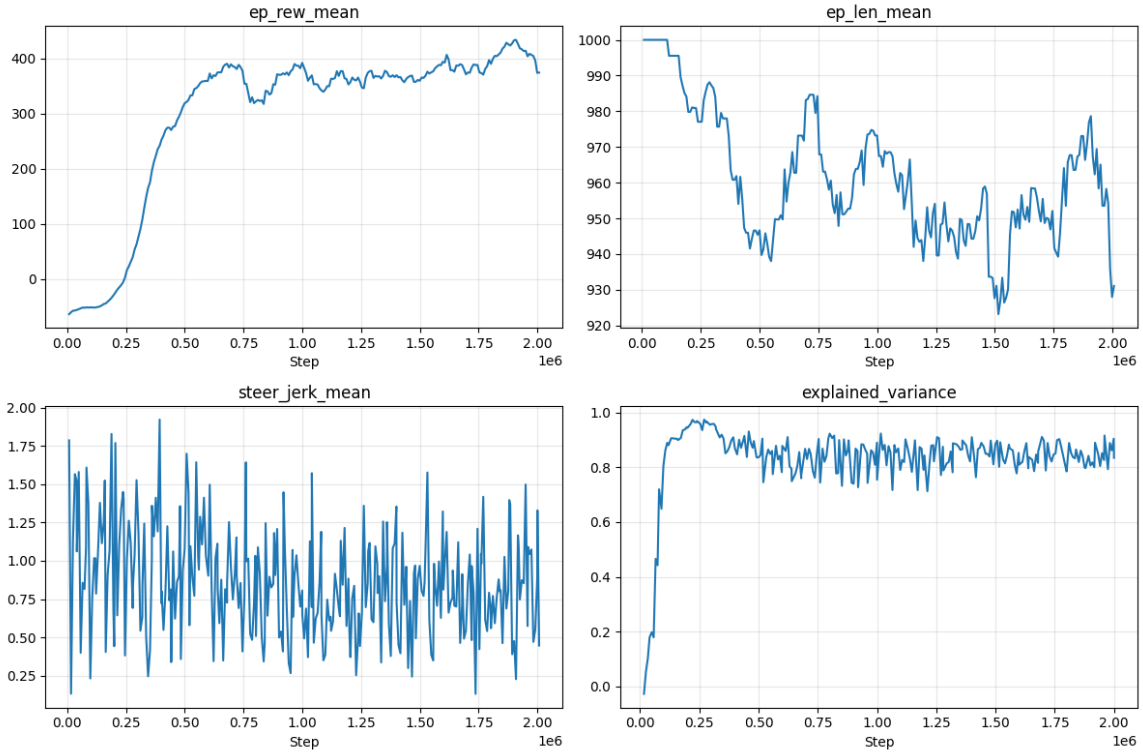

Figure 1: Training curves over 2,000,000 timesteps (TensorBoard).

5.1 Episode Reward (ep_rew_mean)

Reward climbed from approximately −64 at the start of training to a peak of +434, stabilizing in the high 300s to low 400s by the end of the run. This is a substantial and consistent improvement, indicating the agent learned to drive productively under the new reward signal rather than crashing or reversing.

5.2 Episode Length (ep_len_mean)

Episode length started at the maximum (1,000 steps) and remained close to that ceiling throughout training, generally in the 930–1,000 range. Combined with the reward increase, this confirms the agent is completing long, productive episodes rather than ending early from crashes or the wrong-direction termination — it is actively racing, not failing fast.

5.3 Steering Jerk (steer_jerk_mean)

This metric remained noisy across the full run (roughly 0.13–1.9) without a clear downward trend. This is a fair result to report honestly rather than reframe positively: it most likely means the mechanical steering-rate limit is already constraining jerk to a bounded range regardless of what the policy prefers, so the learned policy itself may not have internalized smoother steering — the smoothness observed in the demo video is at least partly enforced mechanically, not purely learned.

5.4 Explained Variance (explained_variance)

Explained variance rose from approximately −0.03 at the start to a stable 0.80–0.97 range for the remainder of training. This indicates the value function converged well and PPO's advantage estimates were reliable — a strong technical health signal for the training process itself, independent of the specific driving behavior learned.

6. Demo Video and Qualitative Result

Five evaluation episodes were recorded against the final trained model using a dedicated recording script. Visual inspection confirmed smooth, confident laps: the car brakes gradually approaching corners rather than abruptly, maintains reasonable speed through turns, and no longer exhibits the aggressive, momentum-blind driving seen in last week's model.

7. Summary

Problem: Prior model cornered aggressively and accelerated without regard to momentum, despite reward increasing during training.

Root cause: The reward function only rewarded track progress, with no signal distinguishing safe speed from unsafe speed relative to upcoming curvature.

Fix: Rebuilt the reward function around a physics-based safe-speed model ($v_{max} = \sqrt{(\mu g r)}$) with anticipatory lookahead, added mechanical steering-rate and wrong-direction constraints, and replaced binary braking with a graduated braking-distance-based control law.

Result: Reward improved from −64 to a stable ~400; episode length stayed near maximum throughout; value function converged well (explained variance ~0.9); demo footage confirms smooth, confident driving.

Open item: Steering-jerk metric did not show a clear learned improvement — current smoothness is likely enforced mechanically rather than fully learned by the policy, worth investigating further next week.

Github Repo: https://github.com/samarth3134/RL-Autonomous-Racing-Agent_wk5